

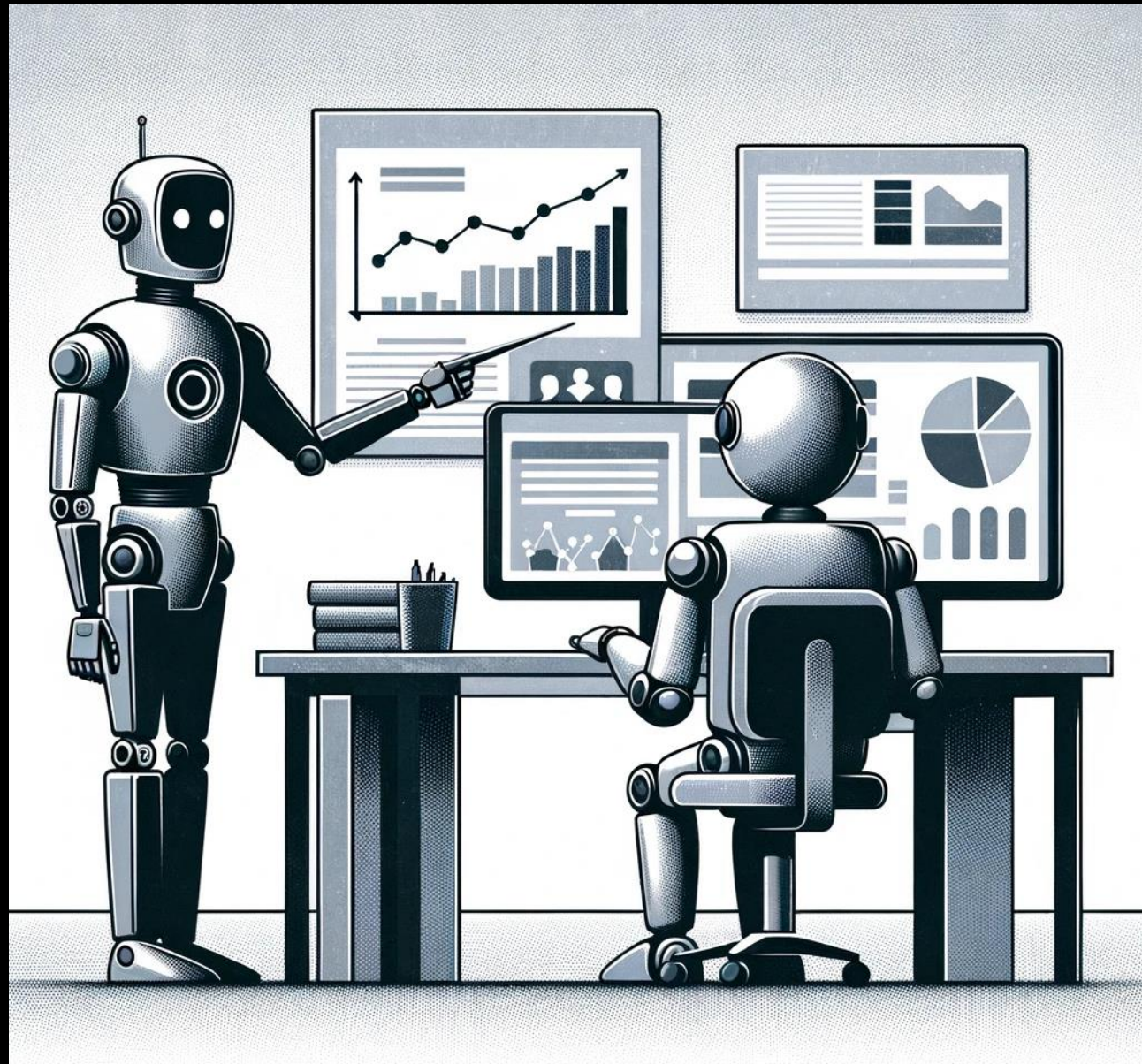
LiDAR 센서와 강화학습

02 강화학습과 자율주행

목차 02 강화학습과 자율주행

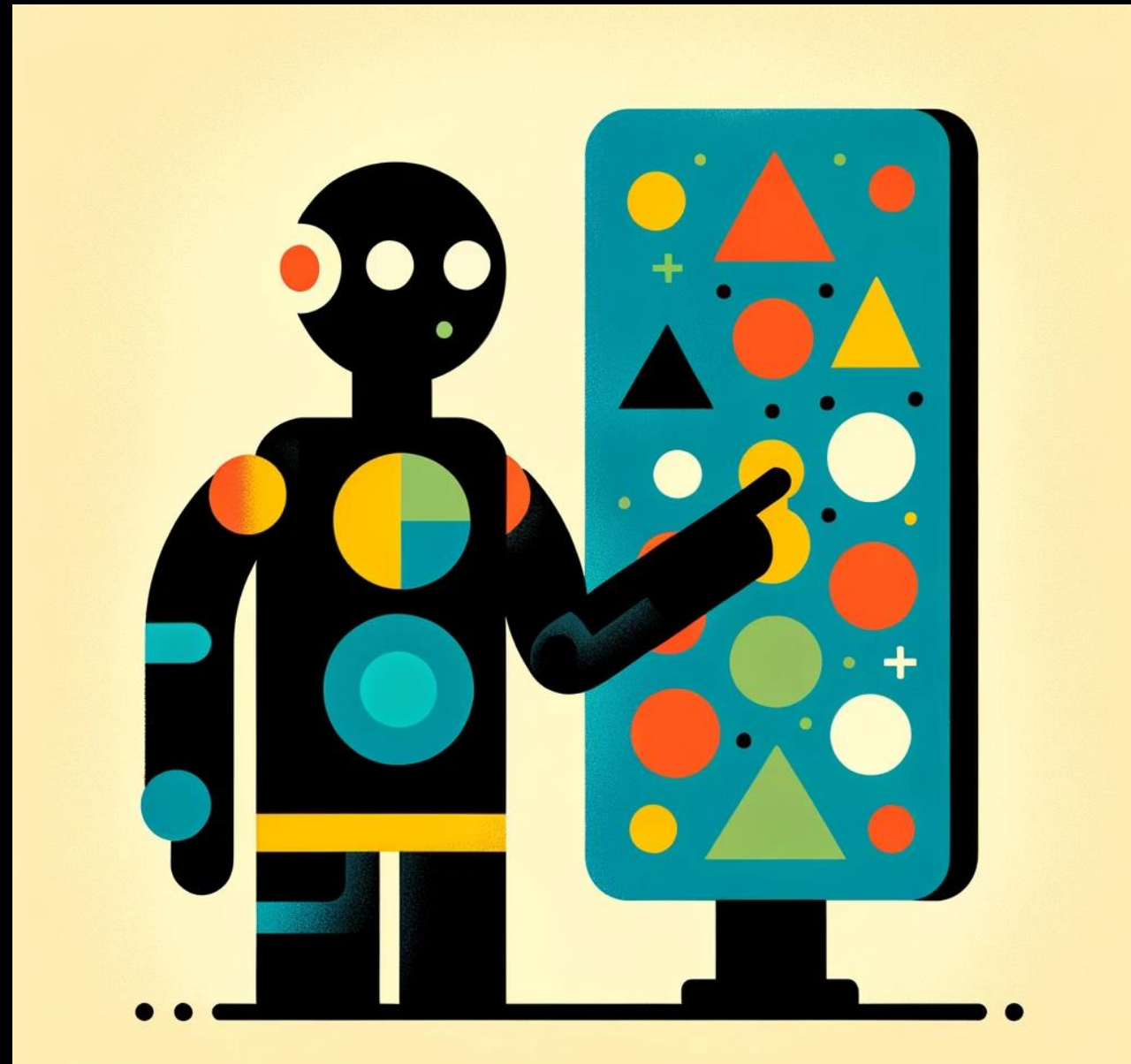
1. 강화학습이란?
2. 강화학습과 자율주행
3. 강화학습 기초

1. 강화학습이란?



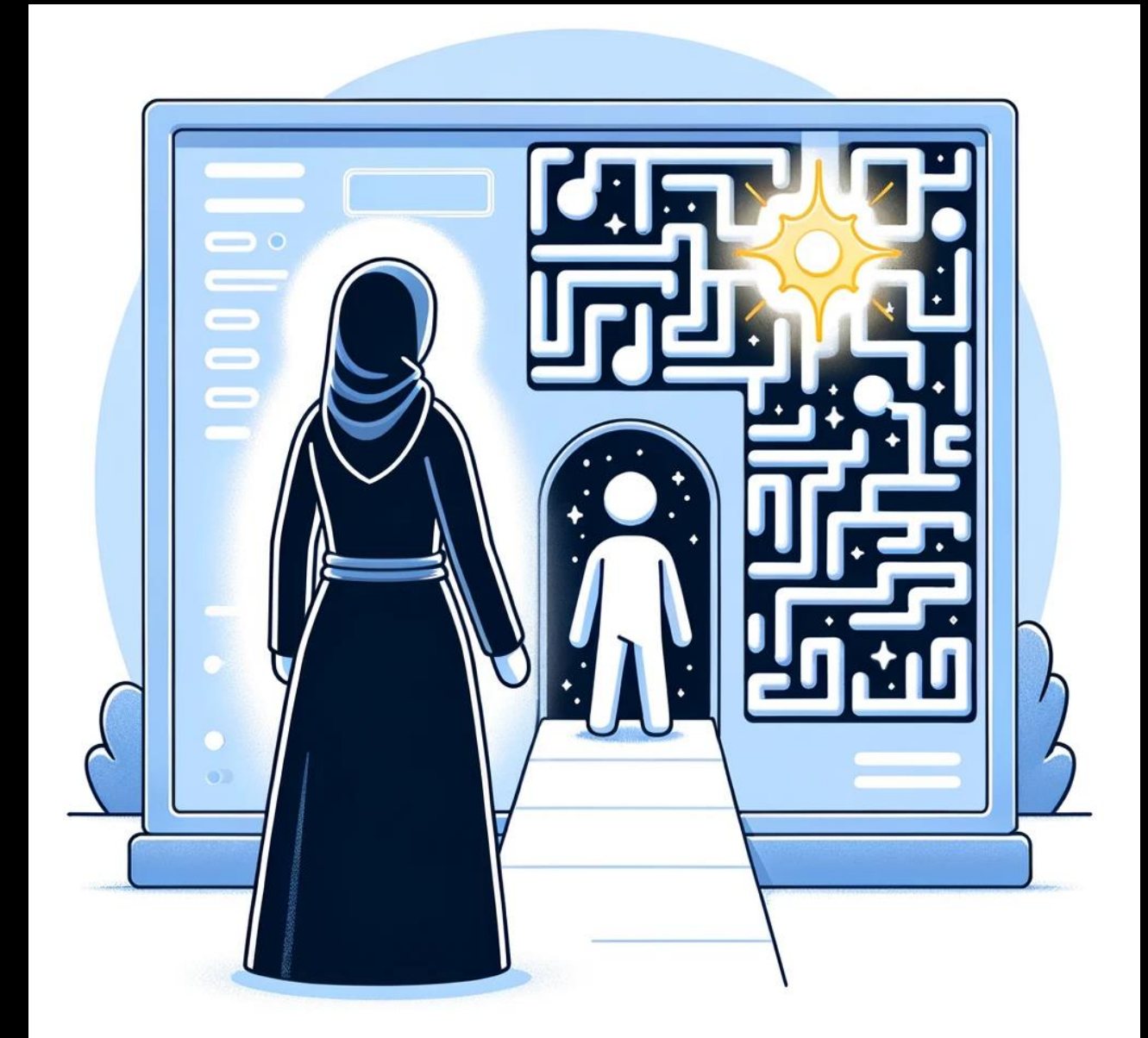
지도학습

AI가 데이터와 정답을 학습



비지도학습

AI가 데이터에서 패턴을 스스로 학습



강화학습

AI가 시행착오를 거쳐 보상을
최대화 하도록 학습

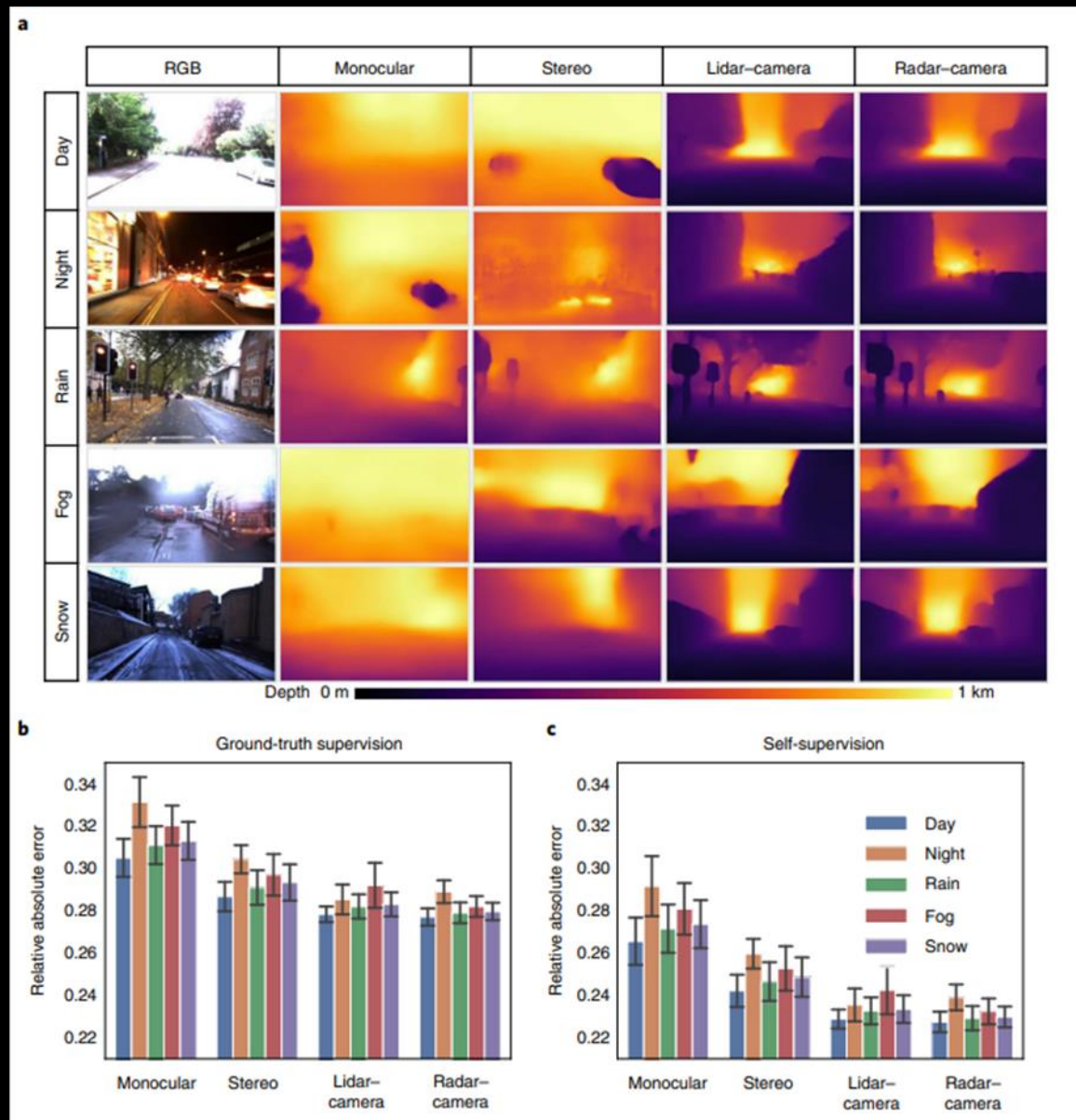


인공지능 학습 기법 적용 예시

LiDAR 센서와 강화학습

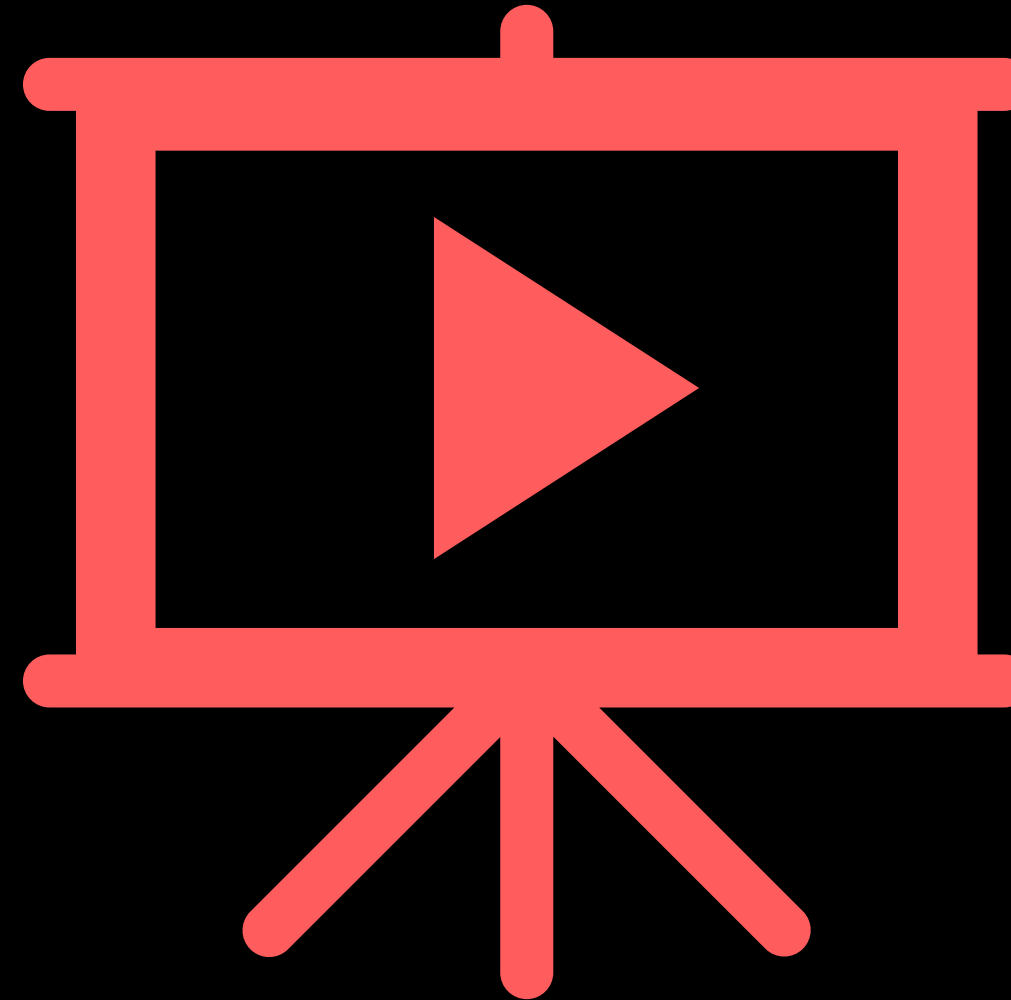
02 강화학습과 자율주행

1. 강화학습이란?



지도학습

차량 주위 날씨 분류 등



비지도학습

유튜브 등에 사용되는 추천 알고리즘



강화학습

스마트팩토리 자물 관리, 자율주행 등

알파고, 알파제로 등도 포함

지도학습: Almalioglu, Y., Turan, M., Trigoni, N. et al. Deep learning-based robust positioning for all-weather autonomous driving. Nat Mach Intell 4, 749–760 (2022). <https://doi.org/10.1038/s42256-022-00520-5>

강화학습: DALL·E 3 으로 생성



주어진 **환경**에서 **시행착오**를 거쳐 환경에서 제공하는 **보상**의 합을 **최대화** 하는 방향으로 AI 모델을 **학습**하는 기법

간략화된 강화학습 도식도



2. 강화학습과 자율주행

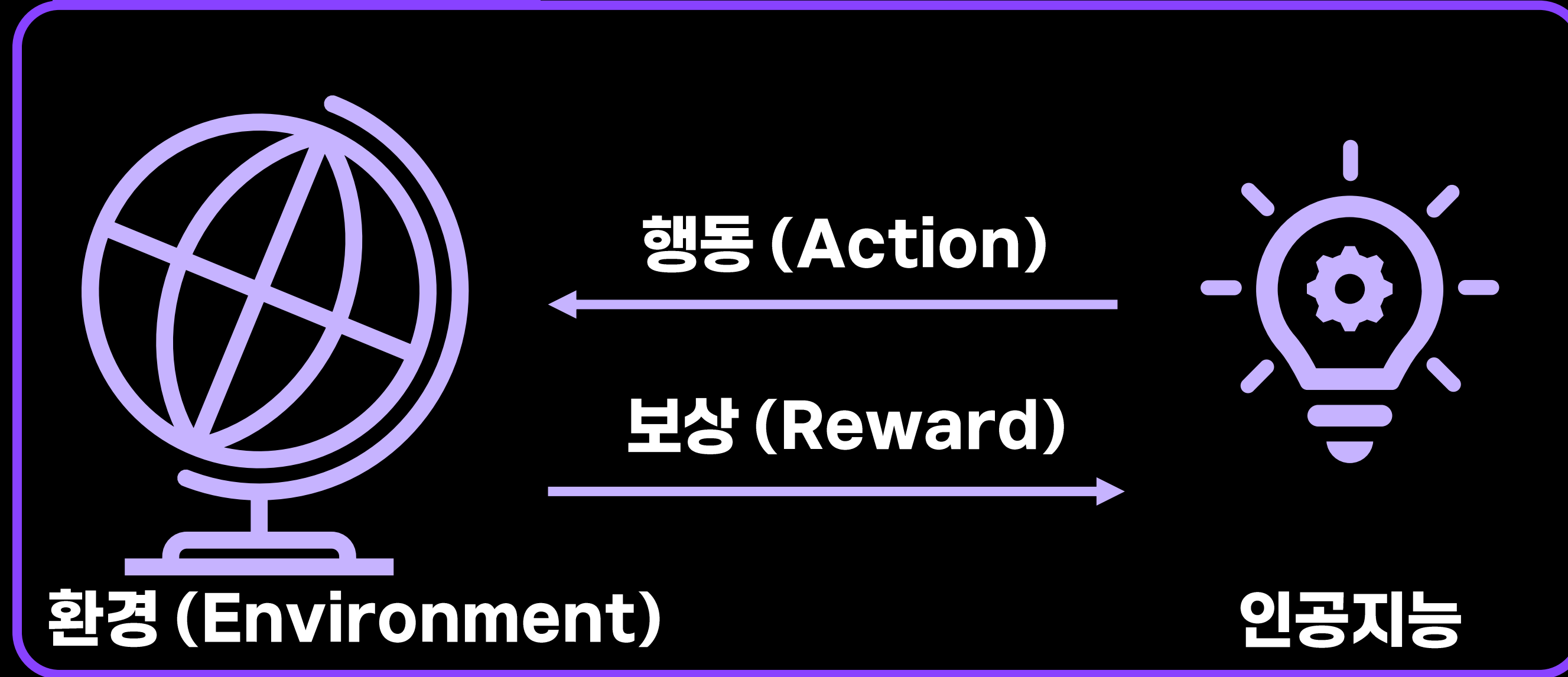


- 에이전트 (Agent) : 환경 정보 (상태) 를 입력 받아 판단하여 행동을 수행하는 주체.
- 상태 (State) : 현재 환경에 대한 정보
- 보상 (Reward) : 에이전트의 행동과 환경 간의 상호작용을 통해 얻을 수 있는 보상
- 행동 (Action) : 에이전트가 주어진 환경에 취하는 행동
- 환경 (Environment) : 에이전트의 행동과 상호작용하여 상태를 바꾸고, 에이전트에게 보상을 제공하는 환경



주어진 환경에서 시행착오를 거쳐 환경에서 제공하는 보상의 합을
최대화 하는 방향으로 AI 모델을 학습하는 기법

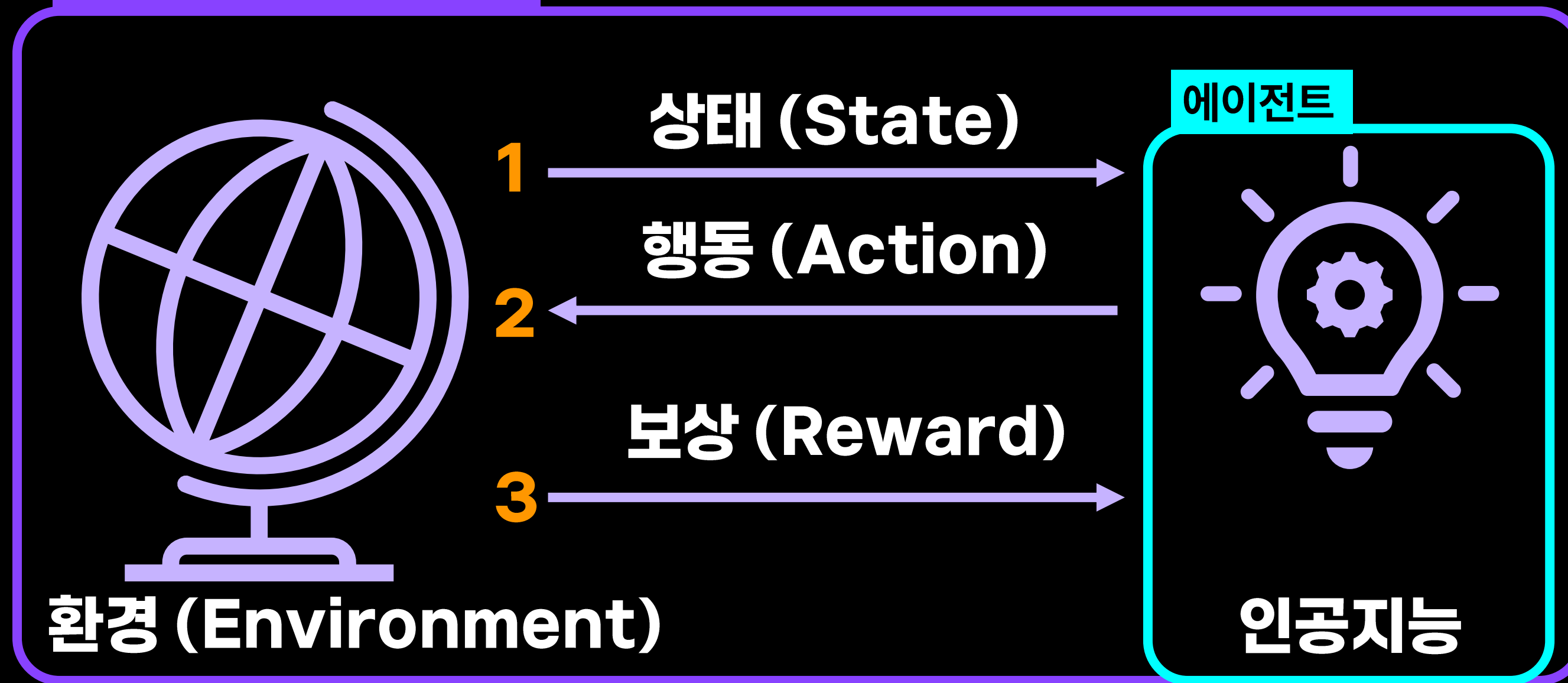
간략화된 강화학습 도식도

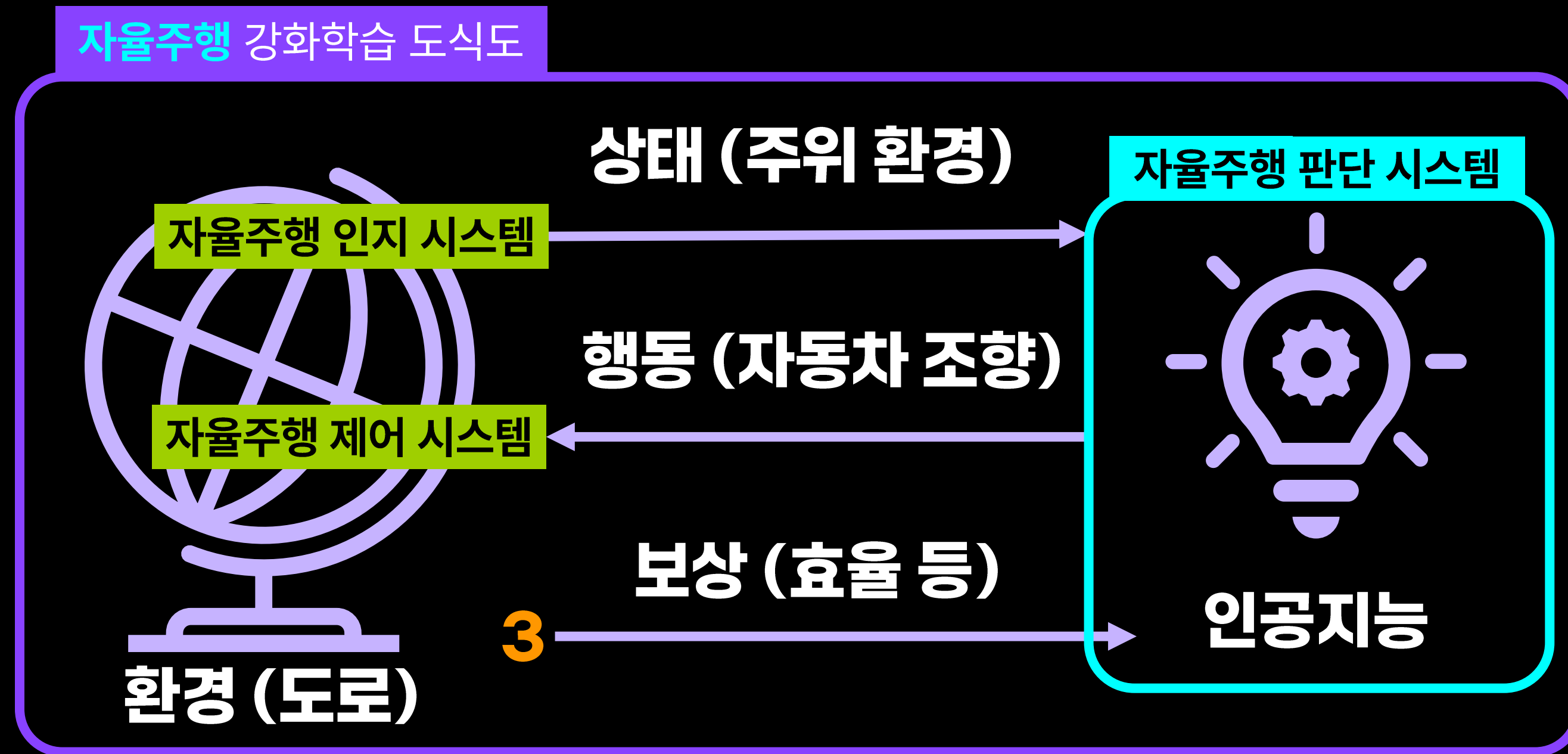


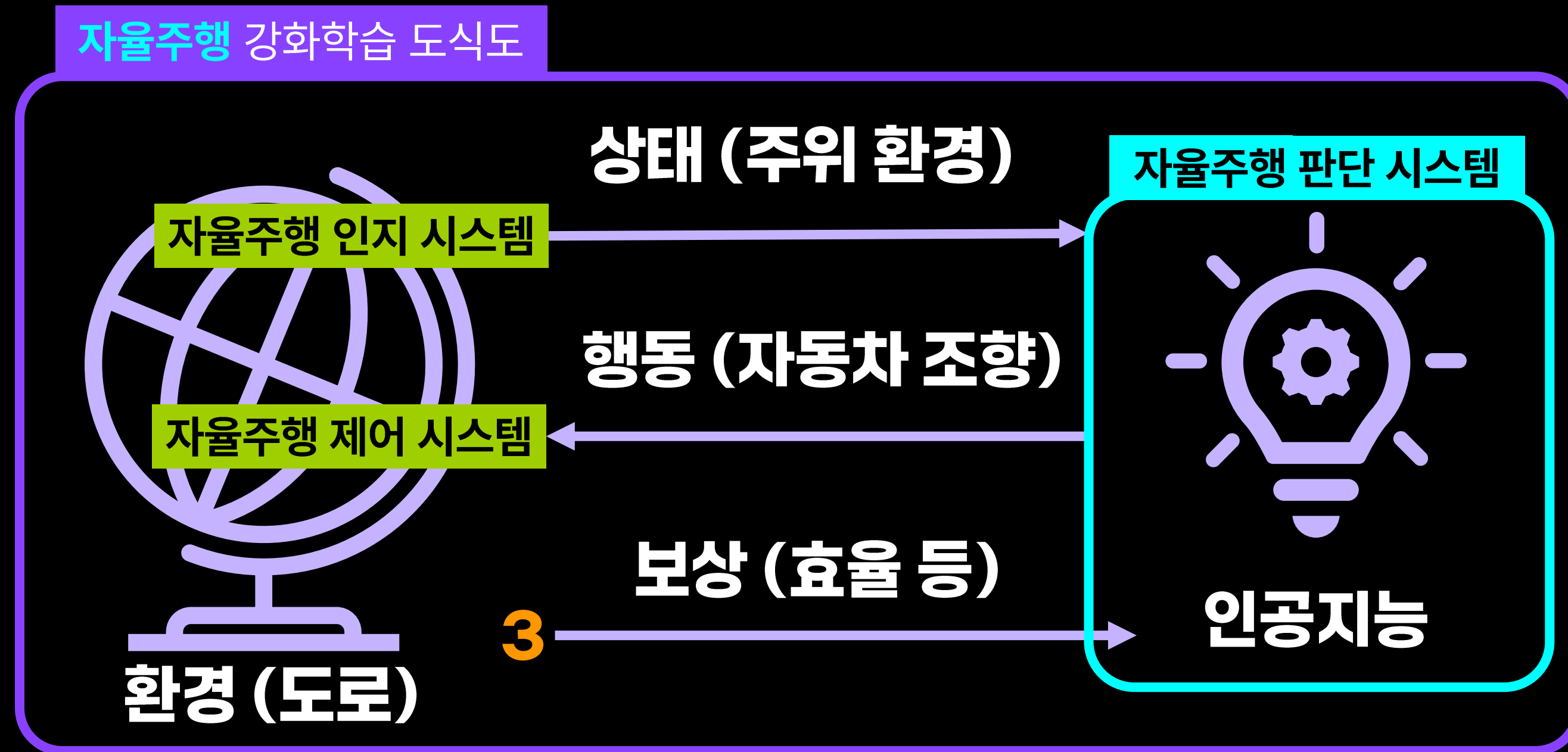


주어진 **환경**에서 **시행착오**를 거쳐 환경에서 제공하는 **보상**의 합을 **최대화** 하는 방향으로 AI 모델을 **학습**하는 기법

간략화된 강화학습 도식도







에이전트 (Agent) : 자율주행 판단 시스템

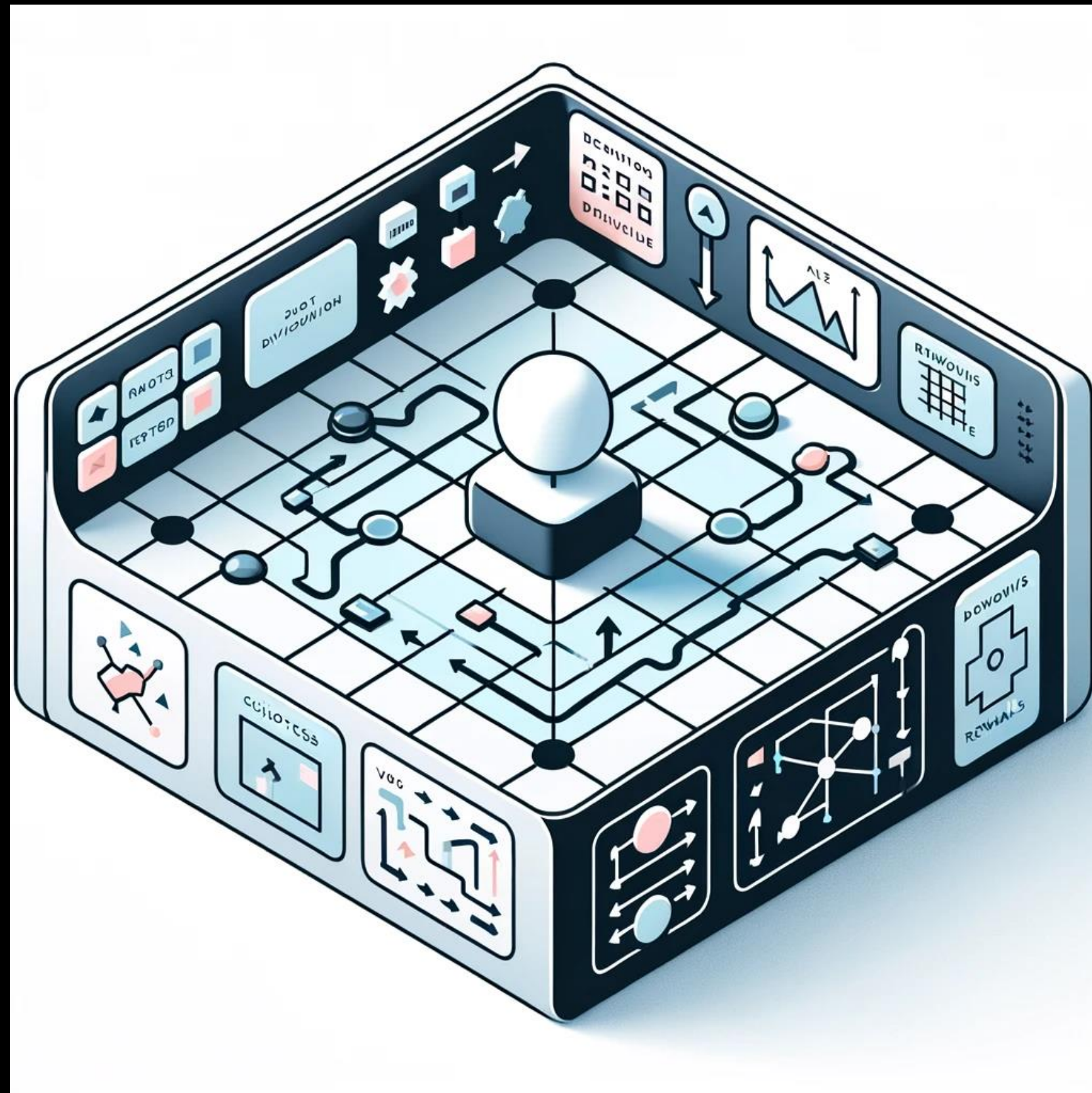
상태 (State) : 자동차 주위 상태

보상 (Reward) : 안전하게 이동한 거리, 효율 등

행동 (Action) : 자동차 가속, 감속, 조향 등

환경 (Environment) : 자동차를 주행하는 도로 환경

3. 강화학습 기초



- **마르코프 결정 과정**은 수학적으로 의사결정을 나타내는 모델
 - 강화학습 문제를 표현할 때 주로 사용하는 모델
- 마르코프 결정 과정 $MDP < S, A, P, R, \gamma >$ 의 구성 요소
 - 가능한 상태의 집합 S
 - 가능한 행동의 집합 A
 - 전이 함수 P
 - 보상 함수 R
 - 감가율 γ
- 강화학습의 구성 요소와 1:1 대응되지 않음에 유의!



- $MDP < S, A, P, R, \gamma >$
- 가능한 **상태의 집합 S** : 주어진 환경에서 가능한 모든 상태를 모아둔 집합
- 자율주행 도메인에서는: 현재 자동차의 위치, 날씨, 주위 교통량, 승객 등 자율주행 환경에서 가능한 모든 상태를 모은 집합.
- 실제로 모든 가능한 상태의 집합을 메모리에 저장하는 것은 아니다.

상태의 예시

현재 상태 s:
주위 환경은 산이며, 주위에
감지되는 차가 없다



- $MDP < S, A, P, R, \gamma >$
- 가능한 **행동의 집합 A** : 주어진 환경에서 에이전트가 취할 수 있는 모든 행동을 모아둔 집합
- 강화학습 모델은 S에 포함된 환경 s를 입력 받아서 A에 포함된 행동 a를 취한다.
- 차체 회전, 기어 변속, 브레이크, 가속 등 자동차가 취할 수 있는 모든 행동의 집합.

행동의 예시

현재 상태 s:
주위 환경은 산이며, 주위에
감지되는 차가 없다

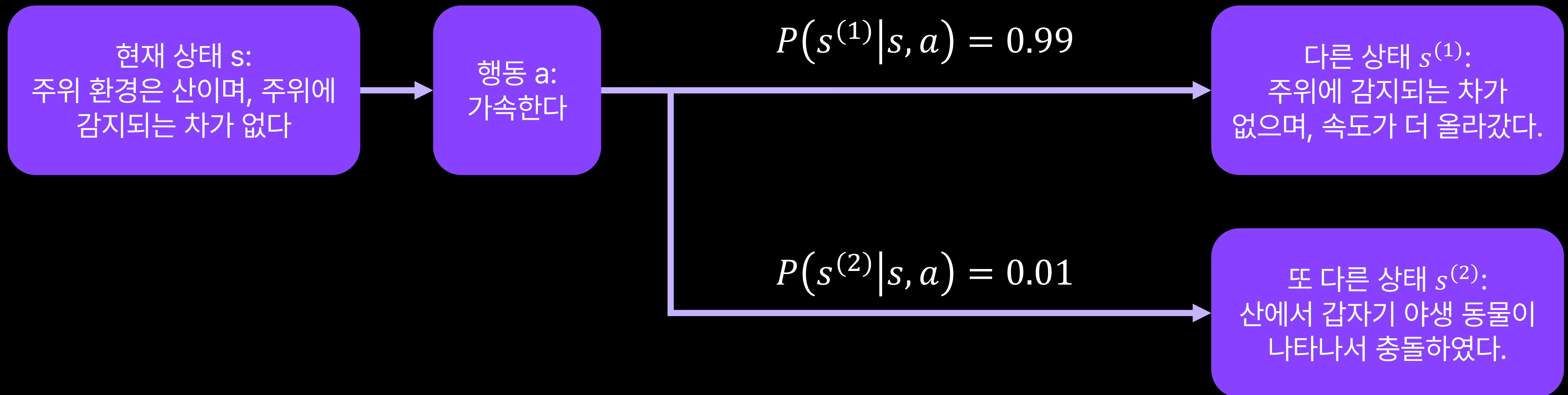


행동 a:
가속한다

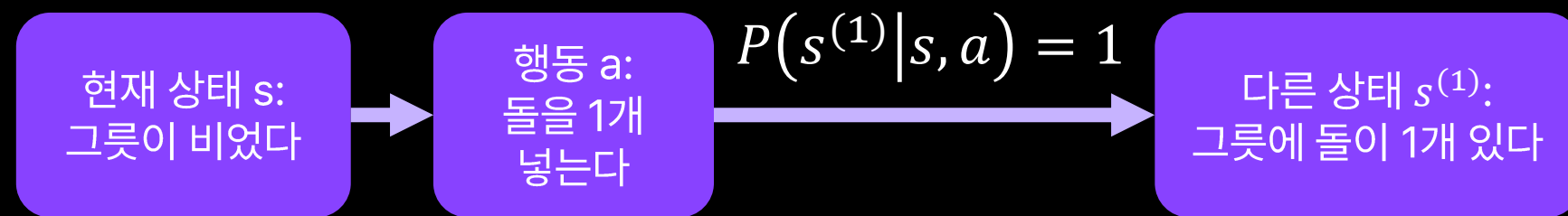


- $MDP < S, A, P, R, \gamma >$
- 전이 함수 P : 특정 상태에서 특정 행동을 취했을 때, 다른 상태로 넘어갈 확률을 나타내는 함수
- $P(s'|s, a)$: 상태 s 에서 행동 a 를 취했을 때, 다른 상태 s' 로 넘어갈 확률

전이함수의 예시

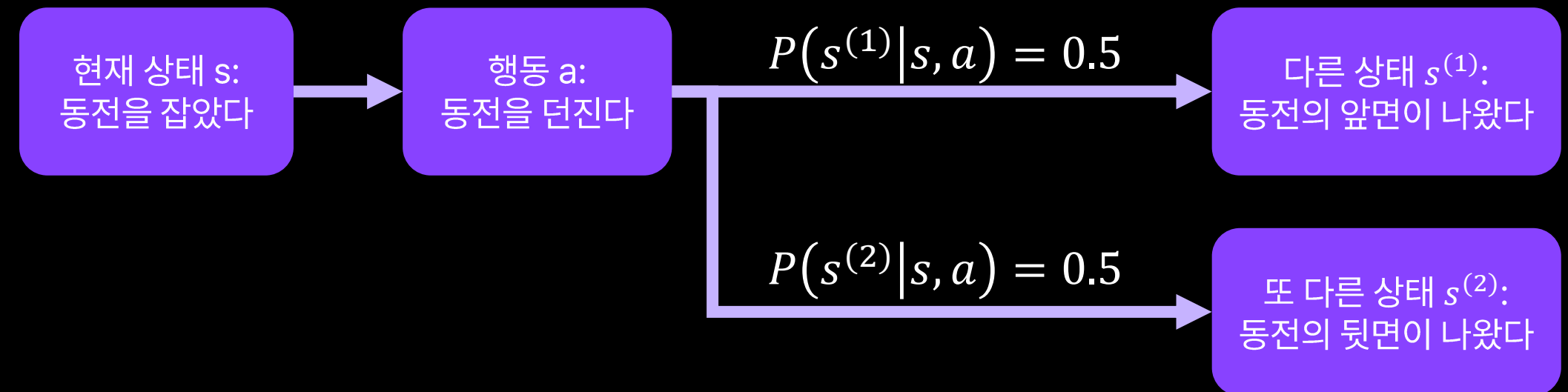


확률 값은 임의로 지정한 값이며, 실제 통계치와는 다릅니다.



결정론적 환경

모든 상태와 행동에 대해,
전이될 상태가 하나로 정해진 환경



비결정론적 환경

일부 상태와 행동에 대해,
전이될 상태가 하나가 아니며 확률에 의해 결정되는 환경



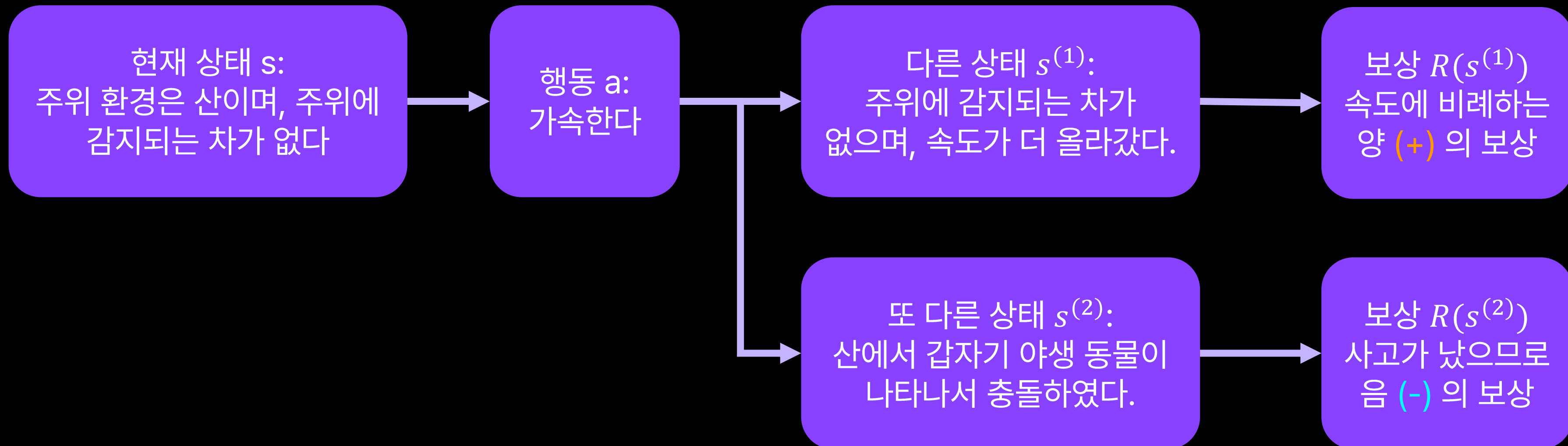
• $MDP < S, A, P, R, \gamma >$

• 보상 함수 R : 특정 상태에 도달했을 때 얻을 수 있는 보상을 나타내는 함수

• MDP 설계 과정에서 보상 함수를 같이 설계하며, 예시는 다음과 같음:

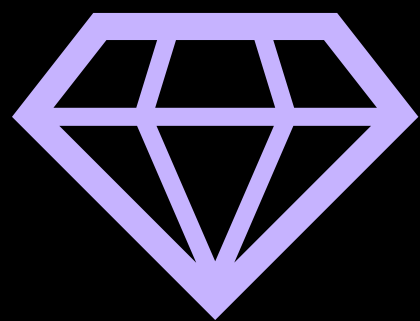
• 예시: 단위 시간동안 빠르게 운행할 수록 그에 비례하는 보상을 주고 사고가 나면 음의 보상 (=페널티)을 제공

보상함수의 예시

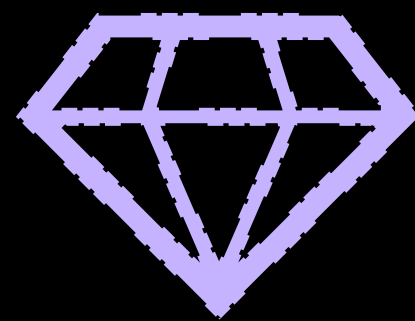




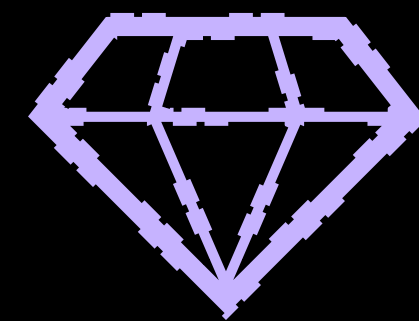
- $MDP < S, A, P, R, \gamma >$
- 감가율 γ (감마) : 현재의 보상 대비 미래의 보상을 중시하는 비율, 0 이상 1 이하
- 미래의 보상을 현재의 보상과 같은 가치로 여긴다면 $\gamma = 1$
- 미래의 보상을 전혀 고려하지 않는다면 $\gamma = 0$



$T = 0$
 $R = 1$



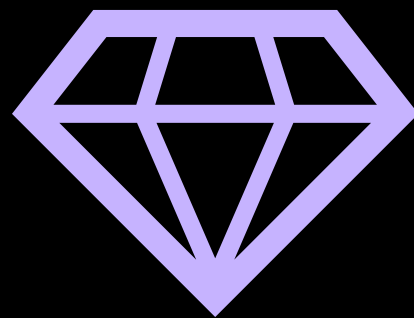
$T = 1$
 $R = \gamma$



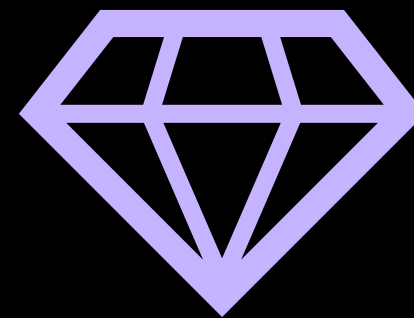
$T = 2$
 $R = \gamma^2$



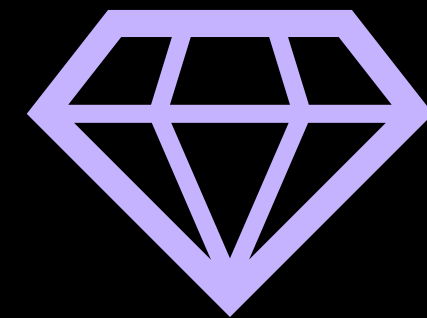
- $MDP < S, A, P, R, \gamma >$
- 감가율 γ (감마) : 현재의 보상 대비 미래의 보상을 중시하는 비율, 0 이상 1 이하
- 미래의 보상을 현재의 보상과 같은 가치로 여긴다면 $\gamma = 1$
- 미래의 보상을 전혀 고려하지 않는다면 $\gamma = 0$

 $\gamma = 1$ 

T = 0
R = 1



T = 1
R = 1

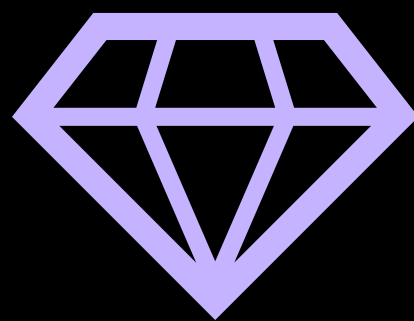


T = 2
R = 1

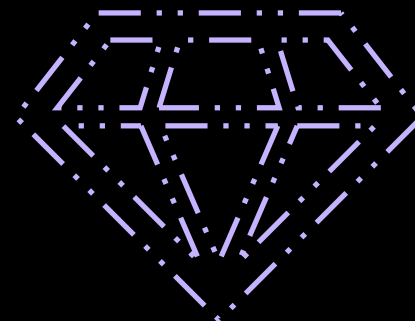


- $MDP < S, A, P, R, \gamma >$
- 감가율 γ (감마) : 현재의 보상 대비 미래의 보상을 중시하는 비율, 0 이상 1 이하
- 미래의 보상을 현재의 보상과 같은 가치로 여긴다면 $\gamma = 1$
- 미래의 보상을 전혀 고려하지 않는다면 $\gamma = 0$

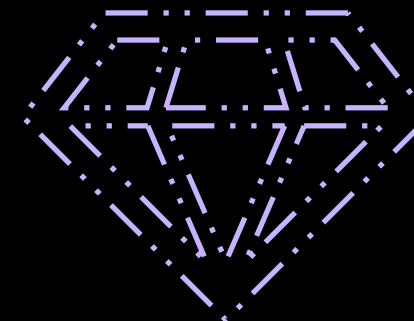
$\gamma = 0$



T = 0
R = 1



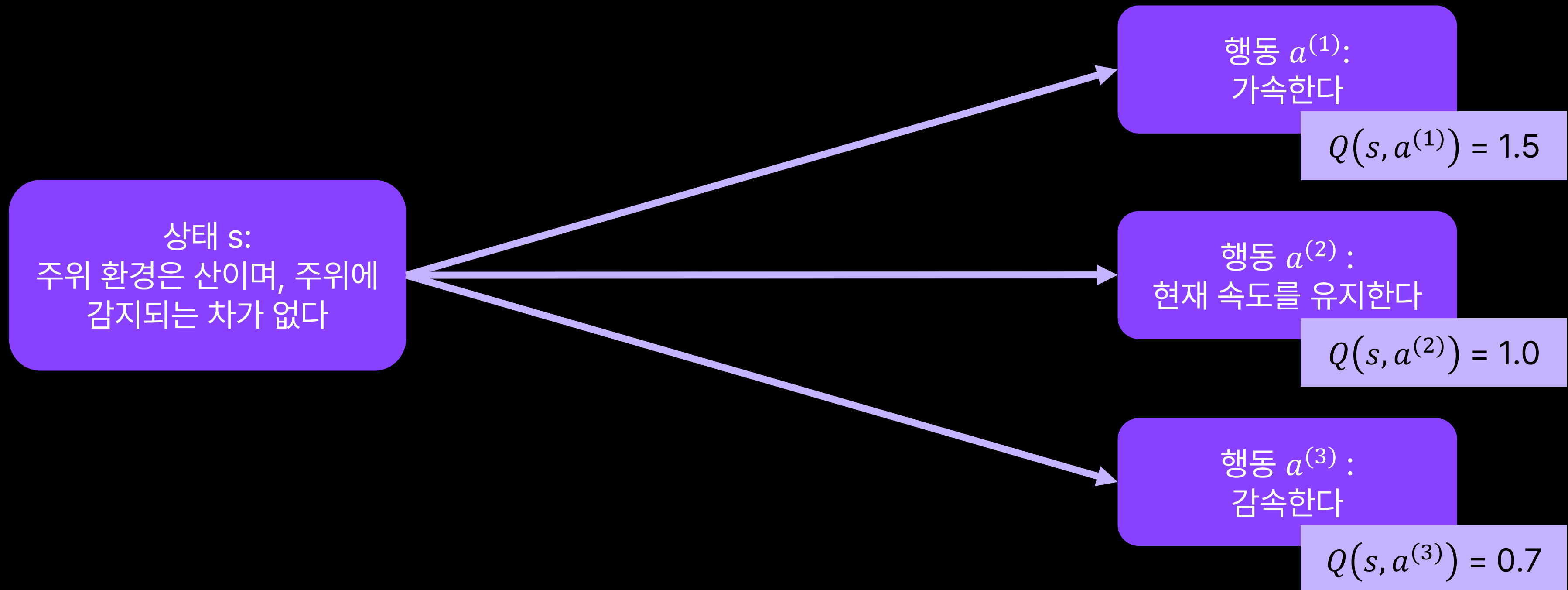
T = 1
R = 0



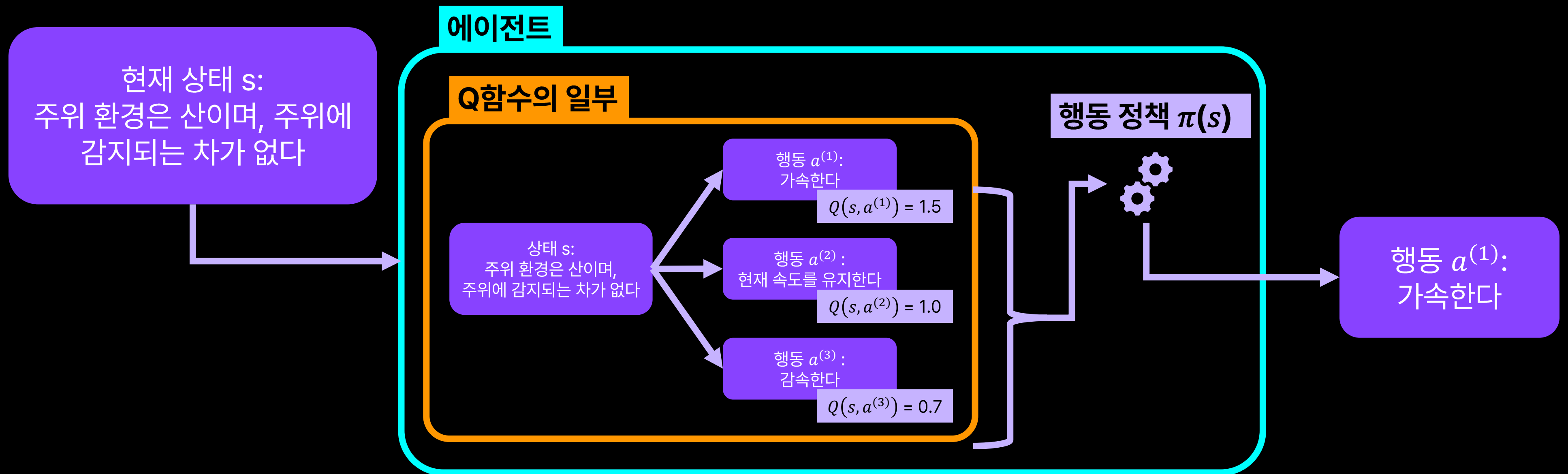
T = 2
R = 0



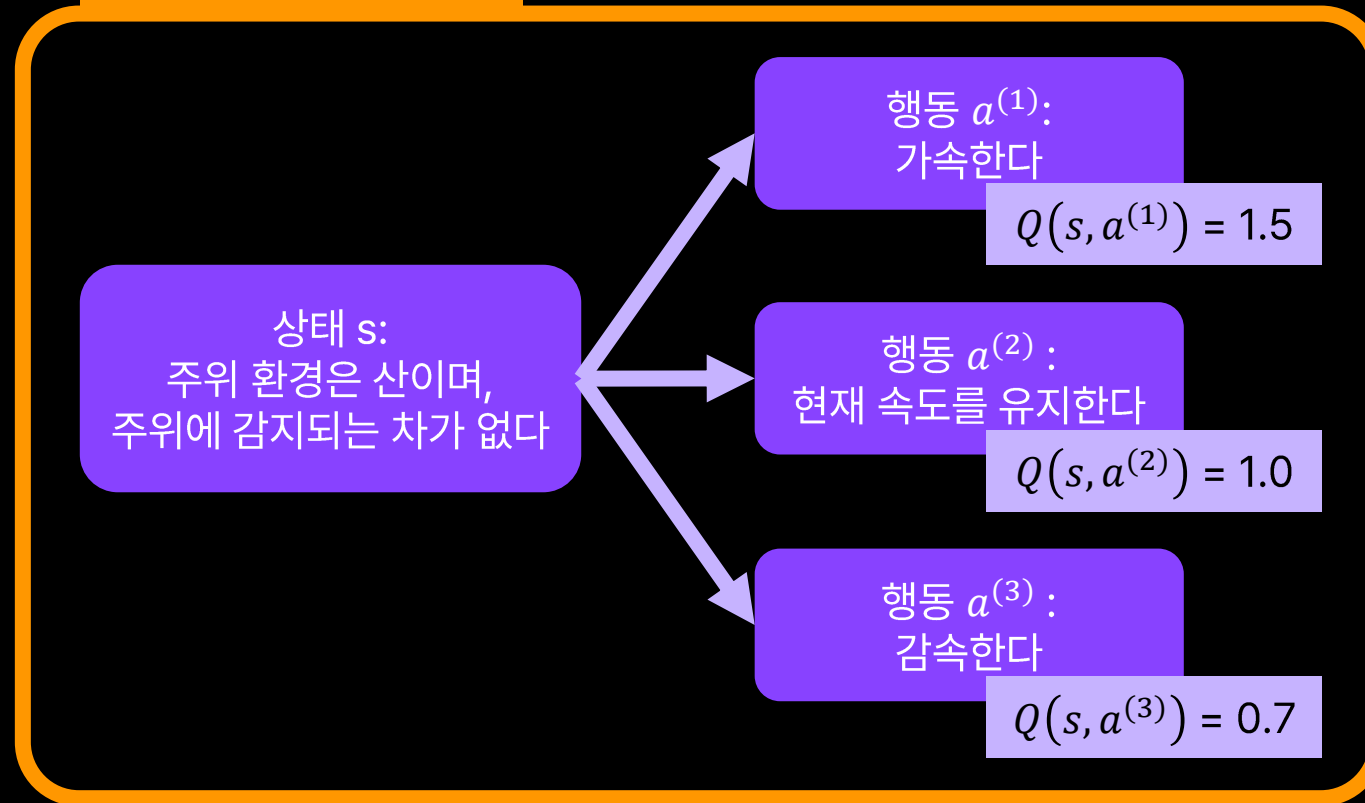
- 현재 상황에서 행동을 취했을 때 **예상**되는 가치를 나타내는 함수, **Q 함수** 라고 부른다.
- $Q(s_t, a_t)$ 는 현재 상태 s_t 에서 행동 a_t 를 취했을 때 얻을 수 있는 **누적 보상의 기댓값**을 나타낸다.
 - $Q(s_t, a_t) = \mathbb{E}[R_t + \gamma R_{t+1} + \dots]$



- **행동 정책** $\pi(s)$ 는 상태 s 에서 취할 행동을 판단하는 함수이다.
- 가치 함수 기반 행동 정책 $\pi(s)$ 는 매 행동마다 **Q 함수의 값이 가장 큰 행동 a** 를 선택한다.
 - $\pi(s) = \operatorname{argmax}_a \{Q(s, a)\}$
 - **에이전트가 알고 있는 Q 함수를 활용한다.**



Q함수의 일부



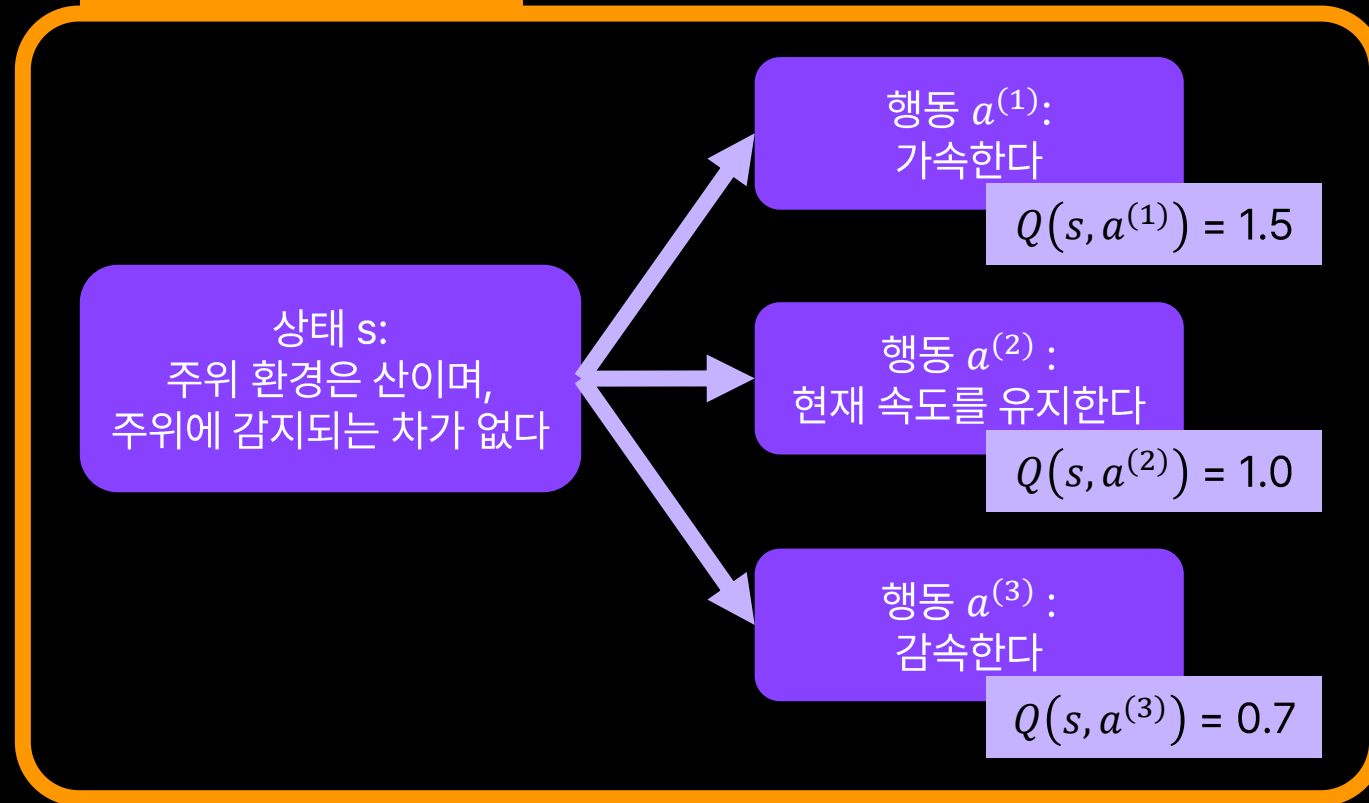
목적: 행동 정책의 성능 향상을 위해 Q 함수를 학습한다.

- 행동 정책 $\pi(s) = \operatorname{argmax}_a \{Q(s, a)\}$

매 순간 Q 함수를 통해 행동을 선택하고, 보상 정보를 수집한다.

주행이 완료된 후 환경이 제공한 보상과 Q 함수로 예측한 보상의 오차를 계산한다. 오차를 기반으로 Q 함수를 업데이트 한다. 이를 반복하여 최적의 Q 함수를 찾는다.

Q함수의 일부



정보 수집

상태 S	선택한 행동 A	얻은 보상 R
$s^{(7)}$	$a^{(1)}$	1.5
$s^{(2)}$	$a^{(13)}$	1.3
$s^{(3)}$	$a^{(7)}$	1.5
$s^{(4)}$	$a^{(2)}$	1.0
$s^{(6)}$	$a^{(3)}$	0.7
$s^{(1)}$	$a^{(1)}$	0.5

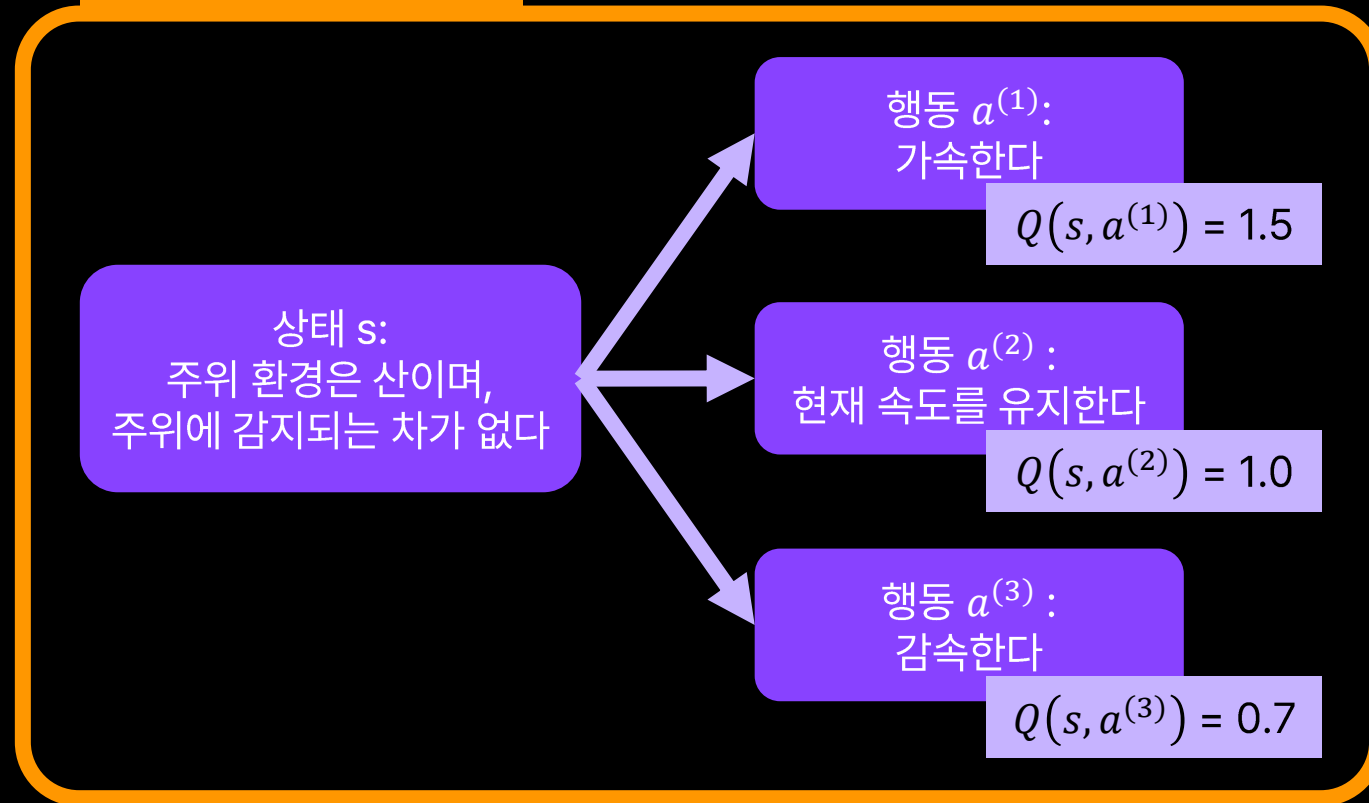
목적: 행동 정책의 성능 향상을 위해 Q 함수를 학습한다.

- 행동 정책 $\pi(s) = \operatorname{argmax}_a \{Q(s, a)\}$

매 순간 Q 함수를 통해 행동을 선택하고, 보상 정보를 수집한다.

주행이 완료된 후 환경이 제공한 보상과 Q 함수로 예측한 보상의 오차를 계산한다.
오차를 기반으로 Q 함수를 업데이트 한다.
이를 반복하여 최적의 Q 함수를 찾는다.

Q함수의 일부



정보 수집

상태 S	선택한 행동 A	얻은 보상 R
$s^{(7)}$	$a^{(1)}$	1.5
$s^{(2)}$	$a^{(13)}$	1.3
$s^{(3)}$	$a^{(7)}$	1.5
$s^{(4)}$	$a^{(2)}$	1.0
$s^{(6)}$	$a^{(3)}$	0.7
$s^{(1)}$	$a^{(1)}$	0.5

목적: 행동 정책의 성능 향상을 위해 Q 함수를 학습한다.

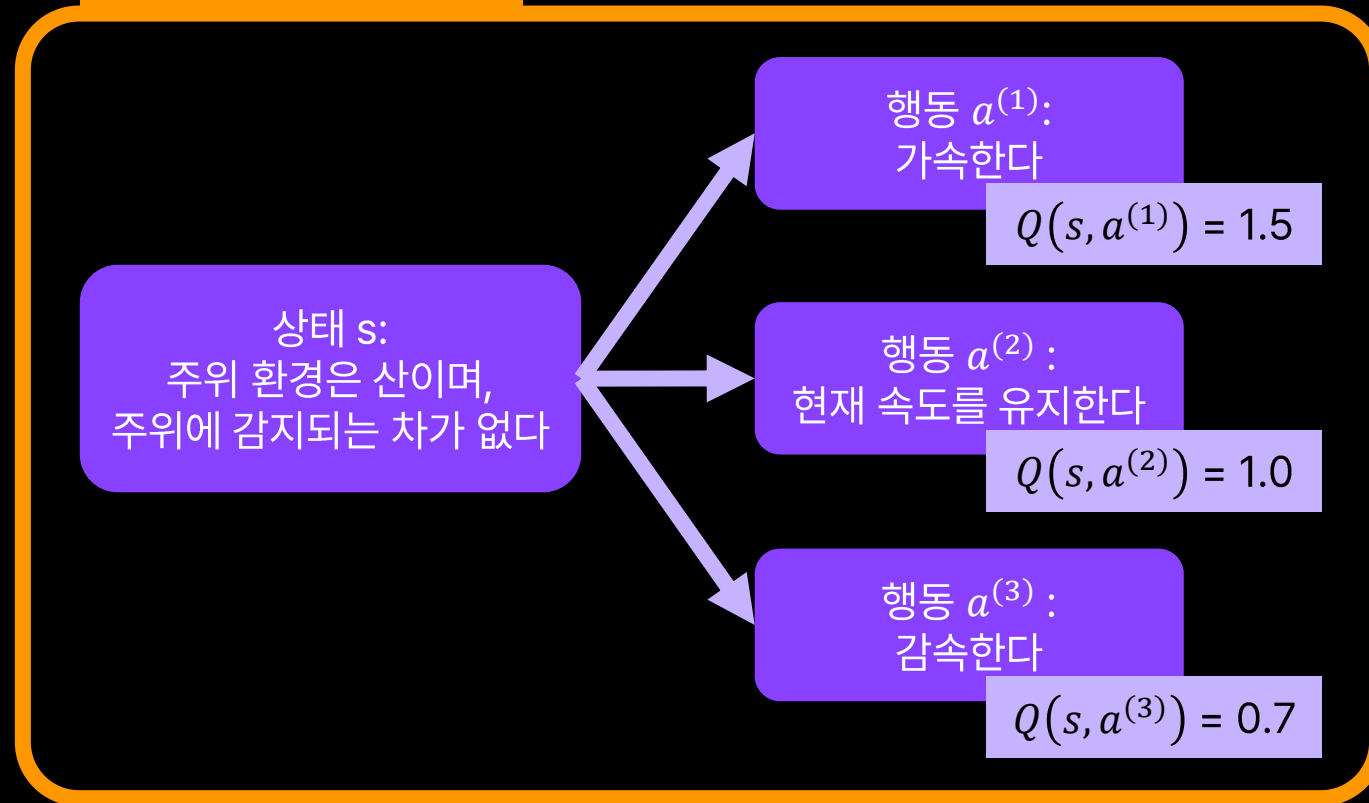
- 행동 정책 $\pi(s) = \operatorname{argmax}_a \{Q(s, a)\}$

매 순간 Q 함수를 통해 행동을 선택하고, 보상 정보를 수집한다.

주행이 완료된 후 환경이 제공한 보상과 Q 함수로 예측한 보상의 오차를 계산한다. 오차를 기반으로 Q 함수를 업데이트 한다. 이를 반복하여 최적의 Q 함수를 찾는다.

$$\text{오차} = \underbrace{R_t + \gamma Q(S_{t+1}, A_{t+1})}_{\text{실제 Q 함수 값}} - \underbrace{Q(S_t, A_t)}_{\text{예측 Q 함수 값}}$$

Q함수의 일부



정보 수집

상태 S	선택한 행동 A	얻은 보상 R
$s^{(7)}$	$a^{(1)}$	1.5
$s^{(2)}$	$a^{(13)}$	1.3
$s^{(3)}$	$a^{(7)}$	1.5
$s^{(4)}$	$a^{(2)}$	1.0
$s^{(6)}$	$a^{(3)}$	0.7
$s^{(1)}$	$a^{(1)}$	0.5

목적: 행동 정책의 성능 향상을 위해 Q 함수를 학습한다.

- 행동 정책 $\pi(s) = \operatorname{argmax}_a \{Q(s, a)\}$

매 순간 Q 함수를 통해 행동을 선택하고, 보상 정보를 수집한다.

주행이 완료된 후 환경이 제공한 보상과 Q 함수로 예측한 보상의 오차를 계산한다. 오차를 기반으로 Q 함수를 업데이트 한다. 이를 반복하여 최적의 Q 함수를 찾는다.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \eta \text{오차}$$

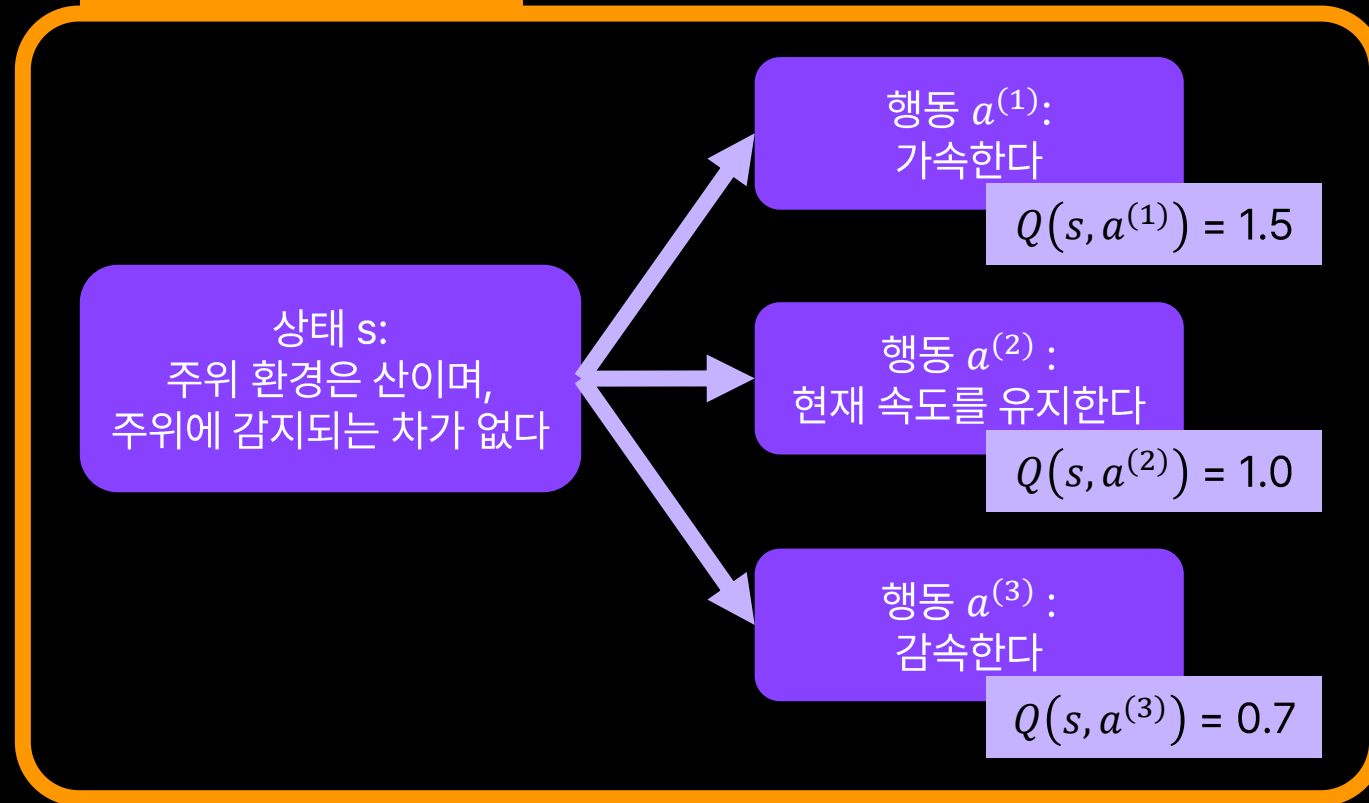
$$\text{오차} = \underbrace{R_t + \gamma Q(S_{t+1}, A_{t+1})}_{\text{실제 Q 함수 값}} - \underbrace{Q(S_t, A_t)}_{\text{예측 Q 함수 값}}$$

실제
Q 함수 값

예측
Q 함수 값

η (에타)는 학습률을 의미합니다.

Q함수의 일부



정보 수집

상태 S	선택한 행동 A	얻은 보상 R
$s^{(7)}$	$a^{(1)}$	1.5
$s^{(2)}$	$a^{(13)}$	1.3
$s^{(3)}$	$a^{(7)}$	1.5
$s^{(4)}$	$a^{(2)}$	1.0
$s^{(6)}$	$a^{(3)}$	0.7
$s^{(1)}$	$a^{(1)}$	0.5

목적: 행동 정책의 성능 향상을 위해 Q 함수를 학습한다.

- 행동 정책 $\pi(s) = \operatorname{argmax}_a \{Q(s, a)\}$

매 순간 Q 함수를 통해 행동을 선택하고, 보상 정보를 수집한다.

주행이 완료된 후 환경이 제공한 보상과 Q 함수로 예측한 보상의 오차를 계산한다. 오차를 기반으로 Q 함수를 업데이트 한다. 이를 반복하여 최적의 Q 함수를 찾는다.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \eta \text{오차}$$

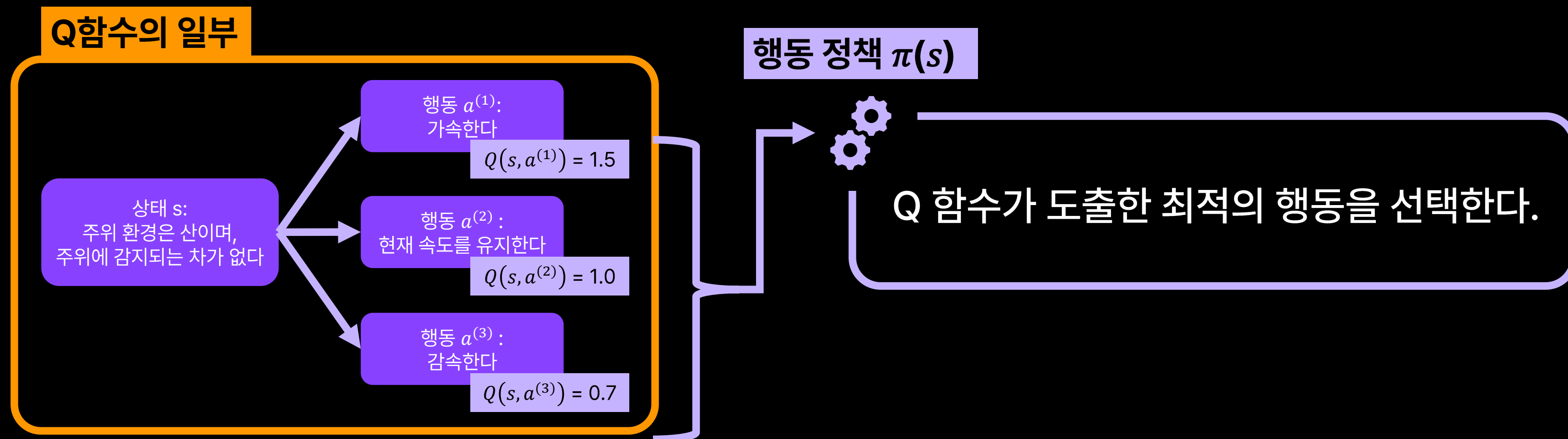
$$\text{오차} = \underbrace{R_t + \gamma Q(S_{t+1}, A_{t+1})}_{\text{실제 Q 함수 값}} - \underbrace{Q(S_t, A_t)}_{\text{예측 Q 함수 값}}$$

실제
Q 함수 값

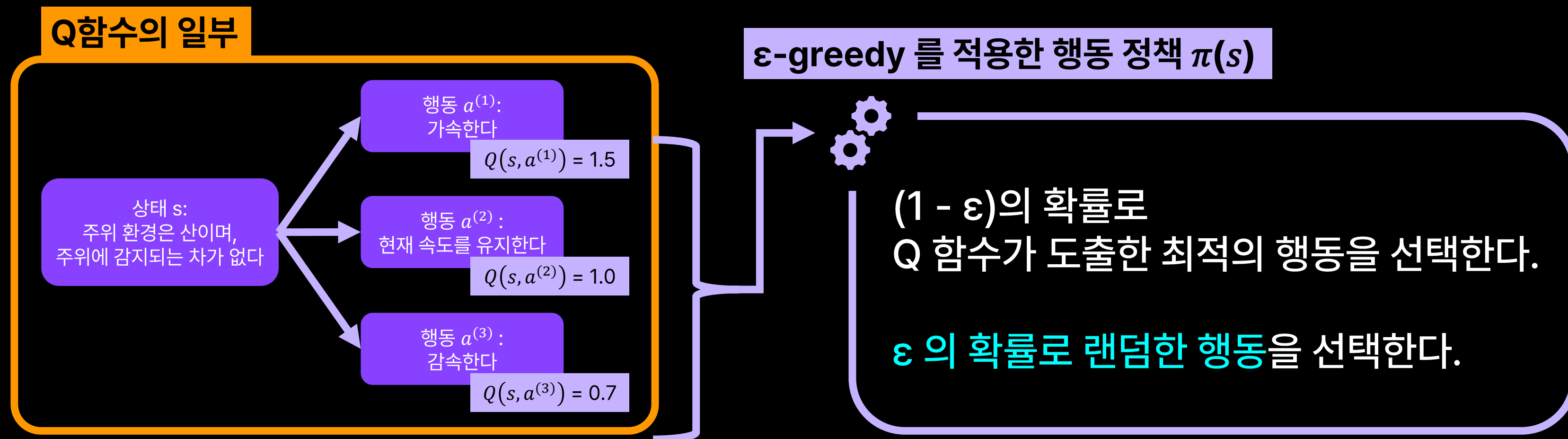
예측
Q 함수 값

η (에타)는 학습률을 의미합니다.

- 목적: 탐험을 장려하여 더 최적의 행동을 찾을 수 있는 가능성을 높인다.
- 모델이 최적해를 찾지 못한 상태에서 고착되는 경우를 방지하고, 탐험을 촉진하기 위한 기법
- 행동 정책이 일정 확률 (ϵ)로 Q 함수가 제시한 최적의 행동이 아니라 랜덤한 행동을 선택한다.

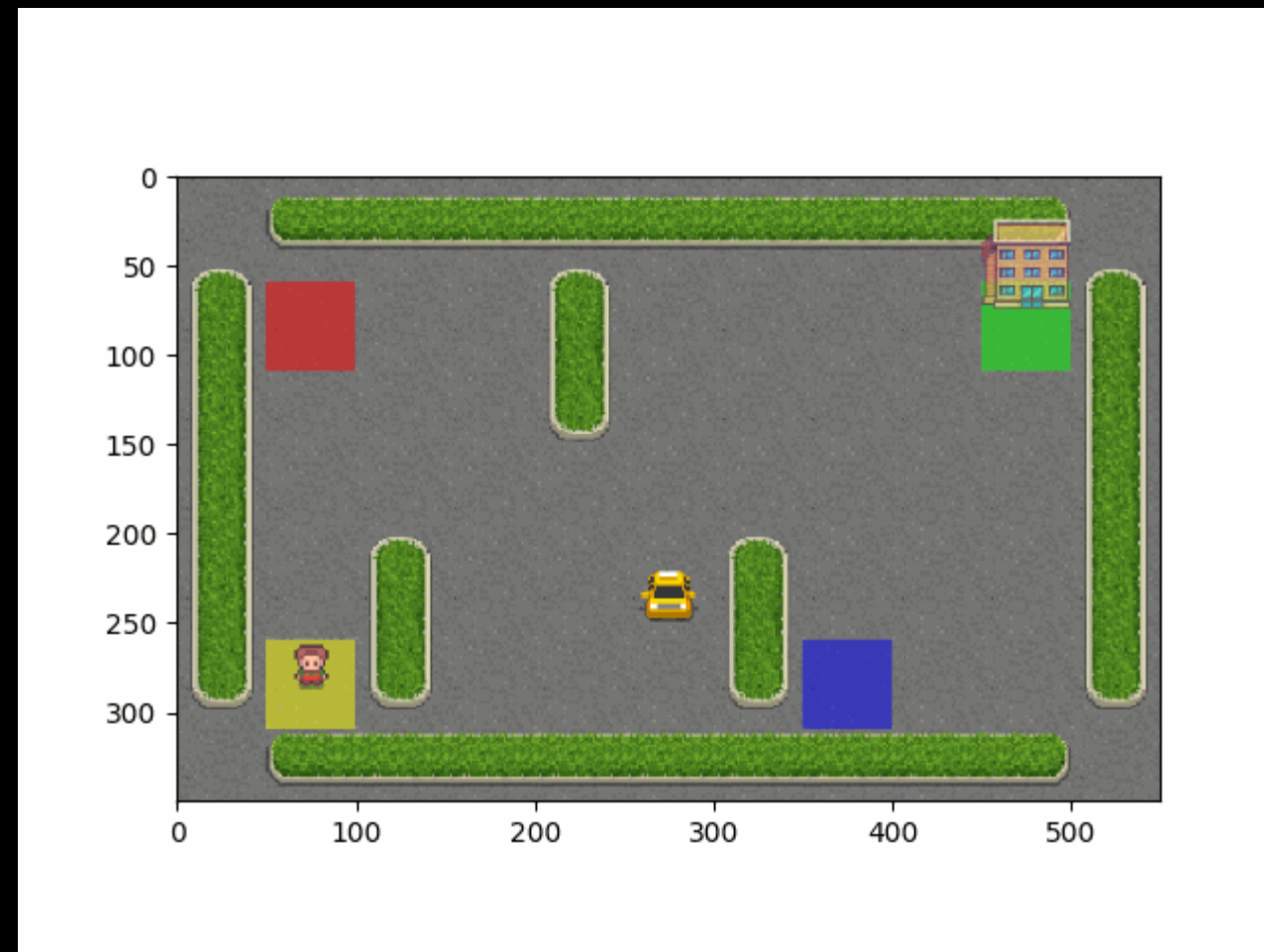
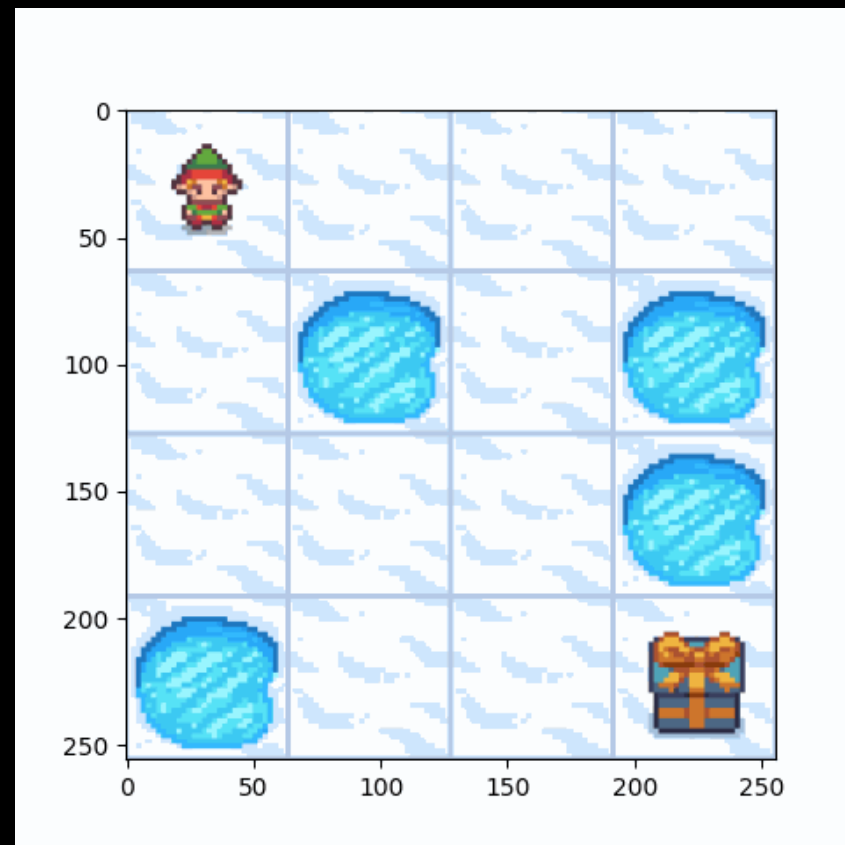


- 목적: 탐험을 장려하여 더 최적의 행동을 찾을 수 있는 가능성을 높인다.
- 모델이 최적해를 찾지 못한 상태에서 고착되는 경우를 방지하고, 탐험을 촉진하기 위한 기법
- 행동 정책이 일정 확률 (ϵ)로 Q 함수가 제시한 최적의 행동이 아니라 랜덤한 행동을 선택한다.





[Q-Learning을 간단한 시뮬레이션 환경에 적용한 모습]



- 2장 실습에서는 간단한 시뮬레이션 환경에 Q-Learning을 사용하여 강화학습의 기초를 실습한다.
- 3장에서 학습할 Q-Learning의 한계로 인해, 실제로는 자율주행에 단순 Q-Learning을 사용할 수는 없다.